



XIAMEN
UNIVERSITY

第五讲 MATLAB编程基础

温程璐 人工智能系



知识要点

- 5.1 MATLAB控制流
- 5.2 函数文件和程序调试



5.1流程控制语句



5.1 流程控制

- 顺序结构
- 判断语句 (if...else...end)
- 分支语句
- 循环语句
- 其他流程控制函数



1. 函数流程控制——顺序结构

- **顺序结构**是最简单的程序结构，系统在编译程序时，按照程序的物理位置顺序执行。
- 这种程序容易编制
- 但是结构单一，能够实现的功能有限。

```
r=1;                                % the radius of the column
h=1;                                % the height of the column
s=2*r*pi*h + 2*pi*r^2;             % calculate the surface area
v=pi*r^2*h;                         % calculate the volume
disp('The surface area of the colume is:'), disp(s);
disp('The volume of the colume is:'), disp(v);
```



2. 函数流程控制——判断语句

- if...end

- 此时的程序结构如下：

if 表达式

 执行代码块

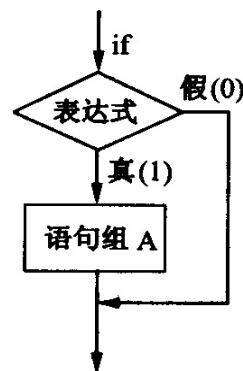
end

- 这是最简单的判断语句，只有一个判断语句，其中的表达式为逻辑表达式，当表达式为真时，执行相应的语句，否则，直接跳到下一段语句。

- 其工作流程如图所示

- 注意：

语句中的end是决不可少的，没有它，在逻辑表达式为0时，就找不到继续执行程序入口。



3. 函数流程控制——判断语句

例子：判断输入的两个参数是否都大于0，是则返回“a and b are both larger than 0”，否则不返回，程序最后返回“Done”。

```
a=input('a=');
```

```
b=input('b=');
```

```
if a > 0 & b>0
```

```
    disp('a and b are both larger than 0');
```

```
end
```

```
disp('Done');
```



3. 函数流程控制——判断语句

- `if...else...end`
- 当程序有两个选择时，可以选择 `if...else...end` 结构，此时程序结构为：

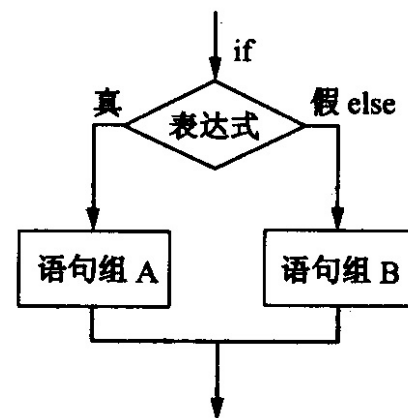
`if 表达式`

 执行代码块1

`else`

 执行代码块2

`end`



- 当判断表达式为真时，执行代码块1，否则执行代码块2。



3. 函数流程控制——判断语句

- 例子1：判断输入的两个参数是否都大于0，是则返回“a and b are both larger than 0”，如果不全大于0，则显示“a and b are not both larger than 0”。

```
a=input('a=');
```

```
b=input('b=');
```

```
if a > 0 & b>0
```

```
    disp('a and b are both larger than 0');
```

```
else
```

```
    disp('a and b are not both larger than 0');
```

```
end
```



3. 函数流程控制——判断语句

- 例子2：计算分段函数的值
程序如下：

```
x=input('请输入x的值:');
```

```
if x<=0
```

```
    y= (x+sqrt(pi))/exp(2);
```

```
else
```

```
    y=log(x+sqrt(1+x*x))/2;
```

```
end
```



3. 函数流程控制——判断语句

- `if...elseif...else...end`
- 上面的两种形式中，分别包含一个选择和两个选择，当判断包含多个选择时，可以采用`elseif`语句，结构为：

`if` 表达式1

 执行代码块1

`elseif` 表达式2

 执行代码块2

`elseif`

.....

`else`

 执行代码块

`end`

- 其中可以包含任意多个 `elseif` 语句。



3. 函数流程控制——判断语句

例1：输入一个字符：

- 若为大写字母，则输出其对应的小写字母；
- 若为小写字母，则输出其对应的大写字母；
- 若为数字字符则输出其对应的数值；
- 若为其他字符则原样输出。



3. 函数流程控制——判断语句

程序：

```
c=input('请输入一个字符','s');  
if c>='A' & c<='Z'  
    disp(setstr(abs(c)+abs('a')-abs('A')));  
elseif c>='a'& c<='z'  
    disp(setstr(abs(c)- abs('a')+abs('A')));  
elseif c>='0'& c<='9'  
    disp(abs(c)-abs('0'));  
else  
    disp(c);  
end
```



3. 函数流程控制——判断语句

例2：判断输入学生成绩的所属等级：60以下不合格，60-89中等，90以上优秀。

```
n=input('input the score:')
```

```
if n>=0 & n<60
```

```
    A='不合格'
```

```
elseif n>=60 & n<89
```

```
    A='中等'
```

```
elseif n>=90 & n<100
```

```
    A='优秀'
```

```
else
```

```
    A='输入错误'
```

```
end
```



4. 函数流程控制——分支语句

- 上一节中的 if...elseif...else...end 语句可以适用于多种选择的流程控制，此时对 else 之外的每一个选择语句设置一个表达式，表达式为真时则执行该模块。
- MATLAB 中的另一种多选择语句为分支语句。分支语句的结构为：
switch 分支语句
 case 条件语句
 执行代码块
 case {条件语句1, 条件语句2, 条件语句3, ...}
 执行代码块
 otherwise
 执行代码块
end



4. 函数流程控制——分支语句

- 其中的分支语句为一个变量，可以是数值变量或者字符串变量，如果该变量的值与某一条件语句相符，则执行相应的语句。
- 否则，执行 otherwise 后面的语句。
- 在每一个条件中，可以包含一个条件语句，可以包含多个条件。当包含多个条件时，将条件以单元数组的形式表示。



4. 函数流程控制——分支语句

例1：任意底对数的实现

```
a=input( '输入底');  
b=input( '输入取对数的值');  
switch a    % 分支语句，可以是变量  
    case exp(1) %条件语句1  
        y = log(b);  
    case 2      %条件语句2  
        y = log2(b);  
    case 10     %条件语句3  
        y = log10(b);  
    otherwise  
        y = log(b)/log(a);  
end
```



4. 函数流程控制——分支语句

- 例2：某商场对顾客所购买的商品实行打折销售，标准如下(商品价格用price来表示)：

price < 200 没有折扣

200 ≤ price < 500 3%折扣

500 ≤ price < 1000 5%折扣

1000 ≤ price < 2500 8%折扣

2500 ≤ price < 5000 10%折扣

price ≤ 5000 14%折扣

输入所售商品的价格，求其实际销售价格。



4. 函数流程控制——分支语句

```
price=input('请输入商品价格');
switch fix(price/100)
    case {0,1}          %价格小于200
        rate=0;
    case {2,3,4}        %价格大于等于200但小于500
        rate=3/100;
    case num2cell(5:9)   %价格大于等于500但小于1000
        rate=5/100;
    case num2cell(10:24) %价格大于等于1000但小于2500
        rate=8/100;
    case num2cell(25:49) %价格大于等于2500但小于5000
        rate=10/100;
    otherwise           %价格大于等于5000
        rate=14/100;
end
price=price*(1-rate)      %输出商品实际销售价格
```



5. 函数流程控制——循环语句——for 语句

- for语句调用的基本格式如下：

for index=初值：增量：终值

语句组A

end

- 功能：把语句组A（亦称为循环体）反复执行N次。循环次数N为：

$$N=1+(\text{终值}-\text{初值})/\text{增量}$$

在每次执行时，程序中的index的值按“增量”增加，for语句的循环次数是预先指定的。



5. 函数流程控制——循环语句——for 语句

例1：用循环求解 $1+2+\cdots+99+100$

```
s=0;
```

```
for i=1:100
```

```
    s=s+i;
```

```
end
```

例2：计算 $\sum_{i=1}^{10000} \frac{1}{2^{i+2}}$

```
s=0;
```

```
for i=1:10000
```

```
    s=s+1/2^(i+2);
```

```
end
```



5. 函数流程控制——循环语句——for 语句

□ for 语句的嵌套

如果一个循环结构的循环体又包括一个循环结构，就称为循环的嵌套，或称为多重循环结构

例子：建立一个10阶数组，数组中的每一个元素 $A(i, j)$ 满足 $A(i, j)=1/(i+j-1)$.

```
for i=1:10
    for j=1:10
         $A(i, j)=1/(i+j-1);$ 
    end
end
```



5. 函数流程控制——循环语句——for 语句

- 注意：当有一个等效的数组方法来解给定的问题时，应避免用for循环。

例如：

例1：用循环求解 $1+2+\cdots+99+100$ 这一问题

```
s=sum(1:100);
```

例2：

```
i=1:10000
```

```
s=sum(1/2.^(i+2));
```



5. 函数流程控制——循环语句——while 语句

- while 语句用于将相同的代码块执行多次，但是次数并不预先指定，当 while 的条件表达式为真时，执行代码块，直到条件表达式为假。

while 语句的结构为：

```
while 表达式  
    执行代码块  
end
```



5. 函数流程控制——循环语句——for 语句

□ 例：用循环求解最小的m，使其满足 $\sum_{i=1}^m i > 10000$

s=0;

m=0;

while (s<=10000)

 m=m+1;

 s=s+m;

end



5.2函数文件和程序调试



M文本文件介绍

MATLAB编写的程序文件称为M文件，M文件有脚本文件和函数文件两种。

- 脚本文件（ M-script ）不需要输入参数，也不输出参数，按照文件中制定的顺序执行命令序列。脚本文件适合于小规模运算。
- 函数文件接受其他数据为输入参数，并且可以返回数据。函数是MATLAB编程的主流方法。



M文本文件介绍

脚本M文件通常仅仅由M文件正文和注释部分构成。正文主要是实现功能，注释则是给出代码说明。

例：

```
g=0: 0.5: 20; %产生 一维向量  
x=sin(g);    % 计算正弦值  
y=cos(g);    % 计算余弦值  
z=[x; y];
```

保存脚本文件时按照MATLAB标识符的要求起文件名

脚本文件的运行有两种方式：

1. 在命令窗口中键入文件名；
2. 在M文件编辑窗口点击Debug菜单的Run，或者使用快捷键F5。



函数M文件介绍

- 函数M文件由function语句引导，格式为：

```
function 输出形参列表 = 函数名 (输入形参列表)  
%注释说明部分 (可选)  
函数体语句 (必须)
```

注意：

1. 第一行为引导行，表示该M文件是函数文件
2. 函数名的命名规则与变量名相同(必须以字母开头)
3. 当输出形参多于一个时，用方括号括起来
4. 函数必须是一个单独的M文件，函数文件名建议与函数名一致，通常为函数名.m,不一致时以文件名为准。



函数M文件介绍

形参

全称为“形式参数” 由于它不是实际存在变量，所以又称虚拟变量。形参是在定义函数的时候使用的参数,目的是用来接收调用该函数时传入的参数。

形参也可以理解为函数的自变量，其初值来源于函数的调用。只有在程序执行过程中调用了函数，形参才有可能得到具体的值，并参与运算求得函数值。

形参在整个函数体内都可以使用，离开该函数则不能使用。



函数M文件介绍

%注释说明部分（可选）

- 第一注释行为大写的函数文件名和函数功能简要描述，供lookfor和help使用
- 第一注释行之后为函数输入/输出参数的含义及调用格式说明等信息，构成全部在线帮助文本
- 在线帮助文本后空一行
- 空一行之后的注释行，包括文件编写和修改的信息，用于软件档案管理



函数M文件介绍

- 编写函数文件计算圆柱体的表面积和体积

`function [s, v]=colume (r, h)`

`% r is the radius of the colume`

`% h is the height of the colume` 注释

`s=2*r*pi*h + 2*pi*r^2;` `% calculate the surface area`

`v=pi*r^2*h;` `% calculate the volume`

注意：

- 当函数具有多个输出变量时，则以方括号括起
- 当函数不含输出变量时，则直接略去输出部分或采用空方括号表示。



函数的调用

- 函数文件不能直接运行，要以函数调用的方式来调用，调用一般格式：

[输出实参列表] = 函数名(输入实参列表)

- 实参必须有确定的值。
- 函数调用时，先将实参传递给相应的形参，从而实现参数传递，然后再执行函数的功能。
- 函数调用时，实参的顺序和个数应与函数定义时的形参的顺序和个数一致。
- 函数调用中发生的数据传送是单向的。即只能把实参的值传送给形参，而不能把形参的值反向地传送给实参。



函数的调用

□ 例:

```
[s, v]=colume (1, 1);
```

或者

```
r=1;
```

```
h=1;
```

```
[s, v]=colume (r, h);
```



函数变量

□ 工作区

MATLAB将每个变量保存在一块内存空间中，这个空间称为工作区。主工作区包括所有通过命令窗口创建的变量和脚本文件运行生成的变量。脚本文件没有独立的工作区，而每个函数都拥有独立的工作区，将该函数的所有变量都保存在该独立的工作区中。

根据变量的作用工作区，函数变量主要分为以下两种：

□ 局部变量

□ 全局变量



函数变量——局部变量

局部变量：

- 每个函数都有自己的局部变量，这些变量存储在该函数 独立的工作区中，与其他函数的变量及主工作区中的变量分开存储。
- 当函数调用结束时，这些变量随之删除，不保存在内存中。并且，除了函数返回值，该函数不改变工作区中其他变量的值。
- 脚本文件没有独立的工作区，当通过命令窗口调用脚本文件时，脚本文件分享主工作区，当函数调用脚本文件时，脚本文件分享主调函数的工作区。
- 需要注意的是，如果脚本中改变了工作区中变量的值，则在脚本文件调用结束后，该变量的值发生改变。
- 在函数中，变量默认为局部变量。



函数变量——全局变量

全局变量

- 局部变量只在一个工作区内有效，无论是函数工作区还是 MATLAB 主工作区。
- 与局部变量不同，全局变量可以在定义该变量在全部工作区中有效。当在一个工作区内改变该变量的值时，该变量在其他工作区中的变量同时改变。
- 任何函数如果需要使用全局变量，则必须首先声明，声明格式为：

global 变量名1 变量名2

变量名列表中的各个变量用空格隔开，不能用逗号！



函数变量——全局变量

- 定义全局变量是M文件间传递信息的一种手段。
- 全局变量给函数间的数据传递带来了方便，但却破坏了函数对变量的封装，降低了程序的可读性，因而在结构化程序设计中，全局变量是不受欢迎的。
- 特别是当程序较大，子程序较多时，全局变量将个程序调试和维护带来不便，故不提倡使用全局变量。



函数类型

- 主函数
- 子函数
- 嵌套函数



函数类型——主函数

- 通常每个 M 文件中的第一个函数为主函数，主函数可以被该文件之外的其他函数调用
- 而子函数只能被该文件内的函数调用
- 主函数的调用通过存储该函数的 M 文件的文件名调用



函数类型——子函数

- 一个 M 文件中可以包括多个函数，除主函数之外的其他函数称为子函数。
- 子函数只能被主函数或该文件内的其他子函数调用。
- 每个子函数以函数定义语句开头，直至下一个函数的定义或文件的结尾。



函数类型——嵌套函数

一个函数内部可以定义其他的函数，这种内部的函数称作嵌套函数。

- 定义嵌套函数时，只要在一个函数内部直接定义即可。需要注意的是当一个 M 文件中存在嵌套函数时，该文件内的所有函数必须以 end 结尾。

- 例 嵌套函数的结构

```
function x = A(p1, p2)
```

```
...
```

```
    function y = B(p3)
```

```
    ...
```

```
    end
```

```
...
```

```
end
```



函数类型——嵌套函数

- 每个函数中可以嵌套多个函数。
- 多个平行嵌套函数

```
function x = A(p1, p2)
```

```
...
```

```
    function y = B(p3)
```

```
    ...
```

```
    end
```

```
    function z = C(p4)
```

```
    ...
```

```
    end
```

```
...
```

```
end
```



函数类型——嵌套函数

- 多层嵌套函数

```
function x = A(p1, p2)
```

```
...
```

```
    function y = B(p3)
```

```
    ...
```

```
        function z = C(p4)
```

```
        ...
```

```
        end
```

```
    ...
```

```
    end
```

```
...
```

```
end
```

- 在这段程序中，函数 A 嵌套了函数 B，函数 B 嵌套了函数 C。



函数类型——嵌套函数的调用

- 一个嵌套函数可以被下列函数调用：
 - (1) 该嵌套函数的上一层函数；
 - (2) 同一母函数下的同级嵌套函数；
 - (3) 被任一低级别的函数调用。



函数句柄

- 利用函数句柄可以实现对函数的间接操作
- 可以通过将函数句柄传递给其他函数实现对函数的操作
- 也可以将函数句柄保存在变量中，留待以后调用操作
- 函数句柄是通过 @ 符号创建的，格式为：

`fhandle = @functionname。`

例：求解方程 $e^x - 3x = 0$

```
function fx=equation (x)
```

```
fx=exp(x)-3*x;
```

```
>> fzero(@equation, 1) ;
```

```
>> fminbnd(@equation, 0, 1) ;
```



程序的调试

应用程序的错误有两类

□ 语法错误

- » 包括词法或文法的错误，例如函数名的拼写错误、表达式书写错误等。
- » MATLAB能检测出大多数该类错误，给出错误信息，并指出出错的位置。

□ 运行时的错误

- » 程序的运行结果有错误，这类错误也称为程序逻辑错误
- » MATLAB系统对程序逻辑错误无能为力



程序的调试

对于逻辑错误，可采用调试手段来发现

- 将程序执行的中间结果输出到命令窗口，以方便检查；
 - 去掉分号
 - 利用函数 `disp` 显示中间变量的值
- 使用调试菜单 (debug)，通过图形界面操作实现程序调试
 - 单步运行
 - 设置断点



MATLAB调试菜单

MATLAB的M文件编辑器中的Debug菜单提供了全部的调试选项

选项	功能	对应快捷键
Step	下一步	F10
Step In	进入被调用函数内部	F11
Step Out	跳出当前函数	Shift+F11
Continue	执行，直至下一断点	F5
Go until Cursor	执行至当前光标处	无
Set/Clear Breakpoint	设置或删除断点	F12
Set/Modify Conditional Breakpoint...	设置或修改条件断点	无
Enable/Disable Breakpoint	开启或关闭光标行的断点	无
Clear Breakpoints in All Files	删除所有文件中的断点	无
Stop if Errors/Warnings	遇到错误或者警告时停止	无



程序的调试方法

以图形方式调试 MATLAB 程序，可以使用编辑器/调试器。也可以在命令行窗口中使用调试函数。两种方法可互换。

开始调试之前，要确保程序已保存且该程序及其调用的任何文件位于搜索路径或当前文件夹中。

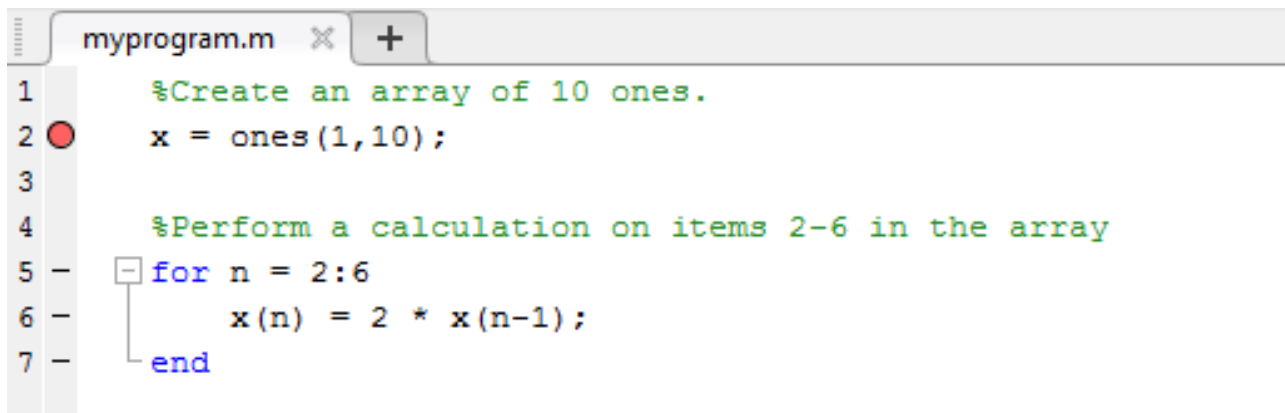
- 如果在未保存更改的情况下从编辑器内运行文件，该文件在运行前会自动保存。
- 如果从命令行窗口运行未保存更改的文件，则 MATLAB 软件会运行已保存的文件版本。因此也看不到更改结果。



程序的调试方法

设置断点

- 设置断点可暂停执行 MATLAB 文件，以便检查认为可能存在问题的值或变量
- 可以使用编辑器、命令行窗口中的函数或同时使用这两种方法设置断点
- 有三种不同类型的断点：标准、条件和错误。要在编辑器中添加标准断点时，在要设置断点的可执行代码行的断点列处点击。断点列是编辑器左侧、行号右侧的窄列。也可以使用 **F12** 键来设置断点
- 断点列中的可执行代码行以虚线 (-) 指示。例如，点击以下代码中第 2 行旁边的断点列，向该行添加断点



```
myprogram.m x +
1      %Create an array of 10 ones.
2  ●    x = ones(1,10);
3
4      %Perform a calculation on items 2-6 in the array
5  -    for n = 2:6
6  -        x(n) = 2 * x(n-1);
7  -    end
```

The image shows a screenshot of the MATLAB Editor. The title bar indicates the file is 'myprogram.m'. The editor contains a script with the following code:

```
1      %Create an array of 10 ones.
2  ●    x = ones(1,10);
3
4      %Perform a calculation on items 2-6 in the array
5  -    for n = 2:6
6  -        x(n) = 2 * x(n-1);
7  -    end
```

A red circle breakpoint is set on line 2. Lines 5, 6, and 7 are indented and preceded by a minus sign (-) in the line number column, indicating they are part of a loop. The 'end' statement is on line 7.



程序的调试方法

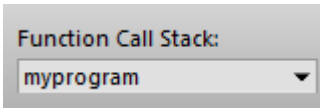
运行文件

在设置断点后，从命令行窗口或编辑器运行该文件。运行该文件会产生以下结果：

- **运行**  按钮更改为 **暂停**  按钮。
- 命令行窗口中的提示符将更改为 K>>，指示 MATLAB 处于调试模式且键盘受控制。
- MATLAB 在该程序的第一个断点处暂停。在编辑器中，断点右侧的绿色箭头表示暂停。在发生暂停的行恢复运行之前，程序不会执行该行。例如，在以下示例中，调试器会在程序执行 `x = ones(1,10);` 之前暂停。

```
1 % Create an array of 10 ones.  
2  x = ones(1,10);
```

- MATLAB 会在 **函数调用堆栈**（位于 **调试** 部分中的 **编辑器** 选项卡上）中显示当前工作区。



如果从命令行窗口使用调试函数，可使用 [dbstack](#) 查看函数调用堆栈。



程序的调试方法

暂停运行文件

要暂停正在运行的程序，请转到**编辑器**选项卡并点击**暂停**  按钮。MATLAB 会在下一个可执行代码行处暂停执行，并且**暂停**  按钮会更改为**继续**  按钮。要继续执行，请按**继续**  按钮。

如果想要检查长时间运行的程序的进度以确保它按预期运行，则暂停很有用。

点击暂停按钮会使 MATLAB 在自己的程序文件外的文件中暂停。按**继续**  按钮会继续正常执行而不更改文件结果。



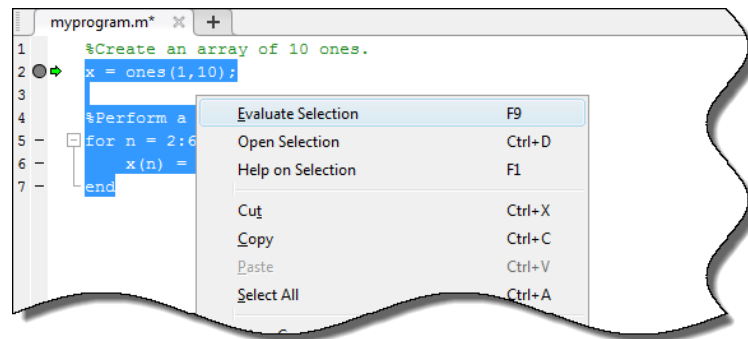
程序的调试方法

调试时修改代码节

可在调试时修改代码段，以在不保存更改的情况下测试可能的修复。通常，最好在退出调试后修改 MATLAB 文件，然后保存修改并运行该文件。

要在调试时修改程序，需要执行以下操作：

1. 代码暂停后，修改尚未运行的文件部分。
断点变为灰色，指示它们无效。



2. 选择 MATLAB 暂停所在行后的所有代码，右键点击，然后从上下文菜单中选择**执行所选内容**。

代码计算完成后，停止调试，保存或撤销所做的任何更改，然后再继续调试过程。



程序的调试方法

逐步执行文件

调试时，可以在要检查值的点暂停，逐步执行 MATLAB 文件。此表介绍了可用调试操作以及可用于执行这些操作的不同方法。

说明	工具栏按钮	备用函数
继续执行文件，直到光标所在行。也可从上下文菜单中获得。	 运行到光标处	无
执行当前文件行。	 步长	dbstep
执行当前文件行，如果该行调用另一个函数，则步入该函数。	 步入	dbstep in
继续执行文件，直到完成或遇到另一断点为止。	 继续	dbcont
步入后，运行被调用函数或局部函数的其余部分，离开被调用函数并暂停。	 步出	dbstep out
暂停调试模式。	 暂停	无
退出调试模式。	 退出调试	dbquit



本章节结束，谢谢。

